IBM Innovation Centre for Education

NCS NihonCommunicationSolutions

Yenepoya Institute of Arts, Science, Commerce & Management.

Project- High Level Design
On
Identity Security Framework for agriculture

Student Name:

Mohammed Thwaha A T P

23BCCE137

Table of Contents

1. Introduction ---------- 1
1.1. Scope of the Document ---------- 1
1.2. Intended Audience ---------- 1
1.3. System Overview ---------- 1
2. System Design ---------- 2
2.1. Application Design ---------- 2
2.2. Process Flow ---------- 3
2.3. Information Flow ---------- 5
2.4. Components Design ---------- 6
2.5. Key Design Considerations ---------- 7
2.6. API Catalogue ---------- 8
3. Data Design ---------- 9
3.1. Data Model ---------- 9
3.2. Data Access Mechanism ---------- 10
3.3. Data Retention Policies ---------- 11
3.4. Data Migration ---------- 11
4. Interfaces ---------- 11
5. State and Session Management ---------- 12
6. Caching ---------- 13
7. Non-Functional Requirements ---------- 14
7.1. Security Aspects ---------- 14
7.2. Performance Aspects ---------- 14
8. Reference ---------- 15

1. INTRODUCTION

The Identity Security Framework for Agriculture is a web-based platform designed to demonstrate secure authentication, authorization, and role-based access control in an agriculture marketplace context. The system, branded as AgriConnect, allows buyers to create accounts and purchase fresh produce, sellers to request approval and upload products, and administrators to approve sellers and monitor users.

The project focuses on the core cyber security difference between authentication, which verifies who a user is, and authorization, which controls what that verified user can do. Passwords are stored in hashed form using Werkzeug security utilities, login is protected with email OTP verification, and user access is separated into Buyer, Seller, and Admin dashboards.

1.1 Scope of the Document

This document covers the high-level architecture, major components, process flow, information flow, database design, user interfaces, session handling, caching approach, and non-functional requirements for the agriculture identity security system. Detailed route-level implementation is covered in the Low-Level Design document.

1.2 Intended Audience

This document is intended for academic evaluators, project guides, mentors, and developers who need to understand the overall architecture and security design of the system.

1.3 System Overview

The system runs as a Flask web application backed by SQLite. Users interact through HTML and Jinja2 templates in a browser. The backend validates credentials, sends OTP codes through Gmail SMTP, creates session state after successful verification, and redirects each user to the correct role dashboard.

```
+----------------------+        +----------------------------+
| Buyer / Seller/Admin | -----> | Web Frontend - Browser     |
+----------+-----------+        | HTML, CSS, Jinja2 Templates |
           |                    +-------------+--------------+
                  |                                |
                  v                                v
 +-----------------------+      +----------------------------+
 | Flask Backend app.py  |----->| Authentication and OTP     |
 | Routes and Controllers|      | Password hash verification |
 +-----------+-----------+      +-------------+--------------+
                    |                                |
                    v                                v
 +-----------------------+      +----------------------------+
 | Authorization Layer   |----->| SQLite database.db         |
 | Buyer/Seller/Admin    |      | users, products, requests, |
 | role checks           |      | activity_logs              |
 +-----------------------+      +----------------------------+
```

Figure 1 - System Overview Diagram

2. SYSTEM DESIGN

2.1 Application Design

The application follows a compact client-server architecture. The browser submits forms to Flask routes, Flask validates the request, reads or writes SQLite records, and returns server-rendered HTML. The design is intentionally simple so students can clearly observe the authentication workflow.

Model	SQLite database.db	Stores users, seller requests, products, and activity logs.
View	Jinja2 HTML templates + CSS	Renders landing page, signup/login forms, OTP page, dashboards, and admin views.
Controller	Python Flask app.py	Handles HTTP routes, validation, database access, file upload, and redirects.
Authentication	Werkzeug security + OTP	Hashes passwords, verifies login credentials, and sends email OTP codes.
Session Layer	Flask signed session / Flask-Login style flow	Maintains temporary OTP state and logged-in user role state.
Email	smtplib + Gmail SMTP	Delivers OTP and seller approval credentials using STARTTLS.

2.2 Process Flow

1. Buyer signup: buyer enters name, email, phone, and password. The system checks password strength and stores the hashed password.
2. Buyer login: buyer enters email and password. If valid, the system generates a 4-digit OTP and sends it to the registered email.
3. Staff login: seller or admin enters user ID and password. If valid, the same OTP verification flow is used.
4. OTP verification: user enters OTP. If it matches the session OTP, the system creates user_id and role session keys.
5. Temporary password reset: approved sellers receive a temporary password and must reset it after first login.
6. Authorization: the session role determines whether the user reaches buyer dashboard, seller dashboard, or admin dashboard.
7. Logout: session.clear() removes authentication and role state.

```
+-------------------------+
| User opens login page   |
+-----------+-------------+
                  |
                  v
+-------------------------+
| Enter credential fields |
+-----------+-------------+
                  |
                  v
+-------------------------+
| Verify hashed password  |
+-----+-------------+-----+
      | invalid     | valid
         v             v
+-----------+   +-------------------------+
| Show error|   | Generate and email OTP  |
+-----------+   +------------+------------+
                                |
                                v
                   +----------------------+
                   | User enters OTP      |
                   +----------+-----------+
                                |
                     +---------+---------+
                     | OTP valid?        |
                     +----+---------+----+
                          | no      | yes
                            v         v
             +----------+  +----------------------+
             | Error    |  | Create session and   |
             | message  |  | redirect by role     |
             +----------+  +----------------------+
```

Figure 2 - Login and OTP Process Flow

2.3 Information Flow

- User credentials flow from the browser form to Flask through HTTP POST.
- Flask queries SQLite using parameterized SQL and retrieves the matching user record.
- Password hash verification is performed using check_password_hash().
- OTP is generated, stored temporarily in session, and sent through Gmail SMTP.
- After successful OTP verification, user_id and role are stored in session.
- Role-based dashboard rendering is controlled through session['role'].
- Seller product images are uploaded to static/uploads and filenames are stored in the products table.

```
 BROWSER          FLASK APP          SESSION STORE        SQLITE DB
                                 SMTP
|                |                   |                  |            |
|--POST login--->|                   |                  |            |
|                |--SELECT user------------------------->|            |
|                |<-------------user row-----------------|            |
|                |--write OTP/temp_user->|                            |
|                |---------------------------------------------->send |
|<--redirect-----|                   |                  |            |
|--POST OTP----->|                   |                  |            |
|                |--read OTP/temp_user>|                |            |
|<--dashboard----|--write user_id/role>|                |            |
```

Figure 3 - Information Flow Diagram

2.4 Components Design

User Interface	Browser-based pages for home, buyer signup, buyer login, staff login, OTP verification, seller request, dashboards, manage users, and activity logs.
Authentication Module	Handles buyer and staff login, password hash checks, OTP generation, OTP verification, and first-login password reset for sellers.
Password Security Module	Enforces strong passwords and stores only hashed passwords using Werkzeug.
Authorization Module	Uses stored session role values to separate Buyer, Seller, and Admin workflows.
Seller Approval Module	Admin approves seller requests, creates seller user IDs, assigns temporary passwords, and emails credentials.

Product Module	Allows approved sellers to upload produce details and product images. Buyers can view all listed products.
Database Module	Stores identity, request, product, and activity data in SQLite.

2.5 Key Design Considerations

- Role separation keeps buyers, sellers, and administrators in different dashboards.
- Passwords are stored as hashes rather than plain text.
- Email OTP adds a second authentication factor after password verification.
- Seller onboarding is controlled by admin approval to prevent unauthorized product listing.
- Uploaded filenames are sanitized using secure_filename() before saving.
- SQLite keeps the project simple to run locally for academic demonstration.
- Session state is cleared on logout to remove active authentication state.
- Parameterized SQL queries are used to reduce SQL injection risk.

2.6 API Catalogue

buyer_signup()	name, email, phone, password	buyer account	Validates password and creates buyer record.
buyer_login()	email, password	OTP redirect/error	Checks buyer credentials and sends OTP.
staff_login()	user_id, password	OTP redirect/error	Checks seller/admin credentials and sends OTP.
verify_login()	OTP	role dashboard	Confirms OTP and creates session keys.
seller_request()	name, email, phone, business	request submitted	Stores seller approval request.
approve_seller(id)	request id	seller account	Creates seller user ID and temporary password.
add_product()	name, price, description, image	product listing	Stores seller product and uploaded image.
logout()	active session	home page	Clears all session data.

3. DATA DESIGN

3.1 Data Model

The system uses a small relational data model. SQLite stores identity records, seller approval requests, product listings, and activity logs. The schema is created automatically by init_db() when the Flask application starts.

```
                    DATABASE SCHEMA - database.db

+-------------------------+      +----------------------------+
| users                   |      | seller_requests            |
| id PK                   |      | id PK                      |
| name, email             |      | name, email, phone         |
| user_id UNIQUE          |      | business_name              |
| password hash           |      | status default pending     |
| role                    |      +----------------------------+
                  | is_temp_password        |
                  +------------+------------+
                                    |
                                | seller_id
                                    v
+-------------------------+      +----------------------------+
| products                |      | activity_logs              |
| id PK                   |      | id PK                      |
| name, price             |      | user_id                    |
| description, image      |      | action                     |
| seller_id               |      | timestamp                  |
+-------------------------+      +----------------------------+
```

Figure 4 - Logical Data Model

users	Identity	id, name, email, user_id, password, role, is_temp_password. Stores buyer, seller, and admin accounts.
seller_requests	Onboarding	id, name, email, phone, business_name, status. Stores pending seller applications.
products	Marketplace	id, name, price, description, image, seller_id. Stores produce uploaded by sellers.
activity_logs	Audit	id, user_id, action, timestamp. Intended for security and user activity tracking.

3.2 Data Access Mechanism

- The sqlite3 module is used for every database operation.
- Each route opens a connection, creates a cursor, executes SQL, commits changes when needed, and closes the connection.
- Queries use ? placeholders for user input to avoid direct string interpolation.
- Password verification is performed in application code after fetching the stored hash.
- Product images are stored on disk while the database stores only the image filename.

3.3 Data Retention Policies

- User accounts remain until manually removed from the database.
- Seller requests are deleted after approval in the current implementation.
- Product records remain until the database file or product row is removed.
- Activity logs are retained for the lifetime of the database. In production, logs older than 90 days may be archived or purged.
- Session data exists only in the active browser session and is cleared on logout.

3.4 Data Migration

Data migration is not required for the academic version. The init_db() function creates the required tables automatically. For a production version, a migration tool such as Flask-Migrate/Alembic should be used to manage schema changes.

4. INTERFACES

Home Page	Landing page	AgriConnect overview with buyer login/signup and seller request links.
Buyer Signup	HTML form	Collects name, email, phone, and password with password strength validation.
Buyer Login	HTML form	Authenticates buyer email/password and triggers OTP verification.
Staff Login	HTML form	Authenticates seller/admin user ID and password.
OTP Verification	HTML form	Accepts OTP sent by email and redirects to role dashboard.
Force Reset	HTML form	Requires approved sellers to replace temporary password after first login.
Buyer Dashboard	Product grid	Displays all available produce listings to buyers.
Seller Dashboard	Form + product grid	Allows sellers to add products and view their own listings.
Admin Dashboard	Dashboard + table	Shows total users, total products, and pending seller requests.
Manage Users	Admin table	Displays registered users for administrative review.
Activity Logs	Admin page	Reserved for audit trail display.

5. STATE AND SESSION MANAGEMENT

The system uses Flask signed-cookie session state to track temporary OTP data and logged-in users. Before OTP verification, the app stores temp_user and otp. After successful verification, it stores user_id and role. These values are used to route the user to the correct dashboard and maintain authentication until logout.

temp_user	Until OTP verification	Temporarily stores the user record after password verification.
otp	Until OTP verification	Stores generated OTP for comparison with submitted OTP.
user_id	Until logout	Identifies the logged-in user or staff member.
role	Until logout	Stores buyer, seller, or admin role for authorization decisions.

- session.clear() is called on logout to terminate active authentication state.
- OTP state should be deleted after successful verification in a production-hardened version.
- The secret key signs session cookies and should be configured through an environment variable in production.

6. CACHING

from Flask and SQLite. Browser caching may apply to static images and CSS, but sensitive pages such as login No server-side caching is implemented in the current academic version. Every page is rendered fresh, OTP, and dashboards should not rely on cache for state.

- OTP values, passwords, and session identifiers are never cached intentionally.
- Static product images can be cached by the browser because they are not authentication secrets.
- A production version may cache public product listings briefly, but user dashboard and admin data should remain uncached.

7. NON-FUNCTIONAL REQUIREMENTS

7.1 Security Aspects

- Passwords are stored using one-way hashes generated by Werkzeug.
- Strong password rules require minimum length, uppercase, lowercase, number, and special character.
- OTP verification provides a second factor after password validation.
- Seller accounts are created only after admin approval.
- secure_filename() reduces unsafe uploaded filename risks.
- Role-based access separates buyer, seller, and admin capabilities.
- SMTP uses STARTTLS for encrypted email delivery.

7.2 Performance Aspects

- The application is lightweight and suitable for local browser demonstration.
- SQLite provides fast lookup for the small academic dataset.
- Server-rendered pages avoid heavy frontend frameworks.
- OTP email delivery may take 1-3 seconds due to SMTP connection time.
- Product images are served as static files for efficient display.
-

8. REFERENCES

- Flask Documentation - https://flask.palletsprojects.com/
- Flask-Login Documentation - https://flask-login.readthedocs.io/
- Werkzeug Security Utilities - https://werkzeug.palletsprojects.com/
- Python sqlite3 Documentation - https://docs.python.org/3/library/sqlite3.html
- Python smtplib Documentation - https://docs.python.org/3/library/smtplib.html
- OWASP Authentication Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html
- OWASP Session Management Cheat Sheet - https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- NIST Digital Identity Guidelines - https://pages.nist.gov/800-63-3/sp800-63b.html